2025 IEEE International Conference on Industry 4.0, Artificial Intelligence, and Communications Technology (IAICT) | 979-8-3315-8649-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/IAICT65714.2025.11100781

CPU Usage Forecasting for Load Balancing in Kubernetes Using LSTM: A Synthetic Traffic Simulation Approach

Amirullah
Department of Informatics Engineering,
Sepuluh Nopember Institute of Technology,
Surabaya, Indonesia
Politeknik Negeri Lhokseumawe,
Lhokseumawe, Indonesia
7025241020@student.its.ac.id, amir@pnl.ac.id

Ahmad Saikhu
Department of Informatics Engineering,
Sepuluh Nopember Institute of Technology,
Surabaya, Indonesia
saikhu@if.its.ac.id

***Abstract*— Kubernetes offers automatic scaling; however, the accuracy of predicting resource requirements remains a challenge in dynamic workload environments. This research proposes LSTM to predict CPU usage in real-time on Kubernetes. The dataset was obtained from e-commerce server logs, and a distribution that identified the beta distribution as the best choice (AIC & BIC). Synthetic data based on the Beta distribution is then simulated using k6, resulting in a time series of CPU usages. The results show that LSTM outperforms ARIMA and GRU with the lowest MSE (0.00001053) and RMSE (0.00324451). The proposed approach can enhance resource allocation efficiency and application stability, as well as provide opportunities to develop real-time workload predictions for more adaptive auto-scaling on Kubernetes.**

Keywords*— Kubernetes, LSTM, auto-scaling, CPU usage, workload prediction, and Beta distribution.

I. Introduction

The transformation of software architectures from monolithic to microservice architectures has become a significant trend in modern application development [1]. This architecture provides greater flexibility and scalability; however, it poses new challenges in resource management as workloads fluctuate. Kubernetes is a popular choice for container orchestration, mainly because of its Horizontal Pod Autoscaler (HPA), which automatically scales pod numbers by evaluating resource metrics like CPU (Central Processing Unit) and memory [2]. However, the effectiveness of this auto-scaling mechanism heavily relies on accurate predictions of resource utilization patterns.

This study utilized the e-commerce server log of zanbil.ir [3], which was processed into 6754 request data points per minute to facilitate analysis. The distribution fitting process indicates that the Beta distribution provides the best fit based on the Akaike Information Criterion (AIC) and the Bayesian Information Criterion (BIC), compared to the Weibull, gamma, log-normal, and normal distributions. Subsequently, synthetic data based on the Beta distribution is simulated using k6 (a modern load testing tool for testing the performance of APIs and microservices) to trigger traffic on the Kubernetes pod. CPU usage is monitored with Prometheus and Grafana, and the time series data were evaluated using ARIMA (Auto Regressive Integrated Moving Average), GRU (Gated Recurrent Unit), and LSTM (Long Short-Term Memory). The results demonstrate that LSTM exhibits the lowest Mean Squared Error (MSE) and Root Mean Squared Error (RMSE), which affirms its superiority in predicting CPU usage.

This finding contributes to improving the automatic scaling efficiency of Kubernetes, which supports microservice architectures. The integration of LSTM-based predictions with HPA facilitates adaptive scaling in real-time, maintaining application stability, and optimizing resource allocation.

II. Related Work

Research on workload prediction in distributed systems has been conducted extensively to improve prediction accuracy and resource management efficiency. For example, [4], [5] explored the use of LSTM in predicting workload patterns in data centers and found that LSTM excels at capturing complex temporal patterns in fluctuating resource usage data. In the context of cloud applications, [6], [7] have emphasized the importance of LSTM's ability to process time-series data to support more accurate resource allocation.

Nevertheless, research focusing on workload prediction in Kubernetes, particularly on integrating predictive models with the Horizontal Pod Autoscaler (HPA), is relatively scarce. Traditional HPA relies solely on CPU and memory metrics, which often inadequately reflect actual needs [8]. Recent studies [9], [10], [11] have indicated that combining predictive models with HPA can significantly enhance resource efficiency through future demand predictions and proactive scaling. Various approaches have been proposed to improve resource utilization, such as Holt-Winter and GRU-based auto-scaling Kubernetes Operators [12], as well as cost-optimization-based schedulers to improve resource utilization [13]. However, the main challenge lies in timely CPU prediction in Kubernetes, especially when facing workload fluctuations [12].

In addition, LSTM has proven effective for continuous workload prediction via real-time data training [14], resulting in high accuracy and low mean squared error [15]. Nevertheless, the application of LSTM to prediction-based automatic scaling in Kubernetes is still limited. Therefore, this research fills this gap by integrating Beta distribution-

based load simulation and the LSTM model to enhance resource allocation efficiency in Kubernetes.

III. METHODOLOGY

This study aims to predict CPU usage based on traffic simulation data generated via random variate Beta distribution and using the Long Short-Term Memory (LSTM) model. The research methodology includes the following steps:

A. Data Collection and Processing

The research dataset originated from the e-commerce web server log [3], which records over 10 million request data entries within a specific period. This data is processed into a representation of requests per minute, resulting in 6754 data entries that reflect the patterns of demand fluctuations. Fig. 1 shows that this dataset provides a rich overview of the demand traffic patterns during the observation period.

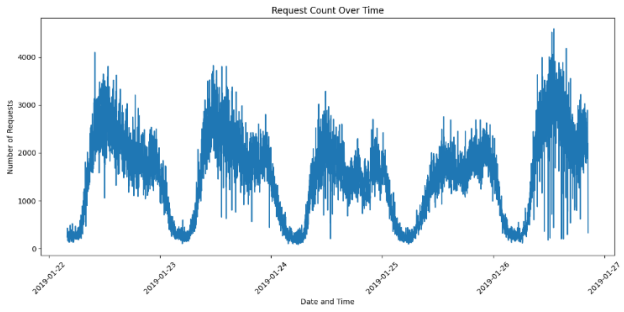

Fig. 1. Research Dataset

The processed dataset consists of two main columns: "datetime," which records the time, and "request_count," which records the number of requests per minute. The recording occurs every minute from January 22, 2019, at 03:56 a.m. until January 26, 2019, at 20:29 a.m. The important statistics from these data include an average of 1.506 requests per minute, a minimum value of 84, a maximum value of 4.590, and a median of 1.594 requests.

B. Distribution Fitting

The process of fitting distributions was performed to determine the model that best represented the pattern of the demand data per minute. Five distributions, namely beta, normal, Weibull, Log-Normal, and gamma distributions, were selected for analysis. The evaluation process applies the Kolmogorov-Smirnov (KS) test to evaluate how well the distribution aligns with the observed data. The Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC) are employed to balance model complexity with data fitting quality. The chosen distribution then serves as a foundation for simulating traffic patterns.

Beta Distribution

$$f(x; \alpha, \beta) = (x^{(\alpha-1)} * (1-x)^{(\beta-1)}) / B(\alpha, \beta) \quad (1)$$

where B(α, β) is the Beta function, and α and β are shape parameters [16] [17].

Normal Distribution

$$f(x; \mu, \sigma^2) = (1/\sqrt{(2\pi\sigma^2)}) * e^{(-(x-\mu)^2/(2\sigma^2))} \quad (2)$$

where μ is the mean, and σ² is the variance [18]

Weibull Distribution

$$f(x; \lambda, k) = (k / \lambda) * (x / \lambda)^{(k-1)} * e^{(-(x / \lambda)^k)} \quad (3)$$

where λ is the scale parameter, and k is the shape parameter.[19], [20]

Log Normal Distribution

$$f(x; \mu, \sigma) = (1/(x\sigma\sqrt{(2\pi)})) * e^{(-(\ln(x)-\mu)^2/(2\sigma^2))} \quad (4)$$

where α α is the shape parameter, β is the rate parameter, and Γ(α) is the Gamma function [24], [25].

Kolmogorov-Smirnov Test

The Kolmogorov-Smirnov test evaluates whether a dataset conforms to a normal distribution, which serves as a fundamental assumption in various statistical analyses.[26].

$$D = \sup_x |F_n(x) - F(x)| \quad (5)$$

The Kolmogorov Smirnov (KS) statistic (D) was used to measure the conformity between the sample data distribution (Fn(x)) and the reference distribution (F(x)). This approach ensures that the selected distribution accurately represents the data for the traffic simulation in Kubernetes.

Akaike Information Criterion (AIC) and Bayesian Information Criterion (BIC)

AIC and BIC contribute to model evaluation and hypothesis testing by providing strict criteria for comparing the quality of various models, considering the balance between model complexity and data fit [27], [28], [29], [30].

$$AIC = 2k - 2\ln(L) \quad (6)$$

Here, k is the number of model parameters, and L is the maximum likelihood of the model. Both are used in the calculation of AIC and BIC to determine the best distribution for the demand data.

$$BIC = k\ln(n) - 2\ln(L) \quad (7)$$

Here, k is the number of parameters in the model, n is the sample size, and L is the maximum likelihood of the model. These three components were used in the calculation of AIC and BIC to evaluate the fit of the distribution with the demand data.

C. Model Evaluation

Three models, ARIMA, GRU, and LSTM, were trained and tested using simulated CPU usage data. The Augmented Dickey Fuller (ADF) test was conducted to verify the stationarity of the data. Model performance was compared using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 scores.

LSTM (Long Short-Term Memory)

LSTM is a Recurrent Neural Network (RNN) designed to process sequential data more accurately and better capture long-term dependencies in the data compared to traditional RNNs. [31].

$$\begin{aligned}&\textit{Forget Gate } f_t = \sigma(W_x \cdot [h_{t-1}, x_t] + b_x)\\&\textit{Input Gate } i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)\\&\textit{Candidate Cell State } \hat{C}_t = \tanh(W_c \cdot [h_{t-1}, x_t] + b_c\\&\textit{Cell State Update } C_t = f_t * C_{t-1} + i_t * \hat{C}_t\\&\textit{Output Gate } o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)\\&\textit{Hidden State } h_t = o_t * \tanh(C_t)\end{aligned} \quad (8)$$

The LSTM model is designed where the sigmoid function ($\sigma(x) = 1/(1+e^{-x})$) and hyperbolic tangent ($\tanh(x) = (ex - e^{-x}) / (ex + e^{-x})$) are used for nonlinear transformation. The weight matrices (W_f,W_i,W_C,W_o) and biases (b_f,b_i,b_C,b_o) are responsible for regulating the learning parameters. The hidden state (ht) and cell state (Ct) represent short-term and long-term memory at time t, while xt is the time step t. The model is configured with an input shape of (5, 1), where a sequence length of 5 is chosen based on autocorrelation analysis that showed dependencies up to 5 lags, using only CPU usage as a feature. The number of hidden units was set to 50, as a compromise between the model's capacity to capture data patterns and prevent overfitting. The batch size was set to 32, which is the standard for deep learning and provides good generalizability while ensuring computational efficiency. This configuration is designed to capture temporal patterns in CPU usage data optimally while considering model efficiency.

GRU (Gated Recurrent Unit)

GRU is another type of RNN that simplifies the LSTM architecture by combining the functions of the forget and input gates into a single update mechanism, thereby reducing complexity and computational cost [32], [33].

$$\begin{aligned}z_t &= \sigma(Wz \cdot [h_{t-1}, x_t] + bz)\\r_t &= \sigma(W_r \cdot [h_{t-1}, x_t] + b_r)\\\hat{h_t} &= \tanh(W \cdot [r_t * h_{t-1}, x_t] + b)\\h_t &= (1 - z_t) * h_{t-1} + z_t * \hat{h_t}\end{aligned} \quad (9)$$

The sigmoid function, represented as ($\sigma(x) = 1 / (1 + e^{-x})$) and the hyperbolic tangent, expressed as ($\tanh(x) = (ex - e^{-x})/(ex + e^{-x})$) are used for nonlinear transformation. The weight matrices (W_z, W_r, W) and biases (b_z, b_r, b) regulate learning, while ht-1 carries previous information, and xt is the current input. The proposed combination effectively captures the temporal patterns of CPU data. The model was configured with an input shape of (5, 1) and 50 hidden units to ensure fair comparison with LSTM. A sequence length of 5 was selected based on the experimental results to provide sufficient capacity to capture complex data patterns.

ARIMA (Auto Regressive Integrated Moving Average)

The ARIMA model is widely used to predict future values based on historical data. This is very useful in cloud computing and data centers to manage resources efficiently [34], [35].

$$\begin{aligned}&yt = c+\phi_1 y_{t-1} +\phi_2 y_{t-2} +\theta_{1\epsilon t-1} +\theta_2 \epsilon_{t-2} +\epsilon_t\\&(1-B)y_t =(1-\phi_1 B-\phi_2 B^2)(1-B)^d y_t=(1+\theta_1 B+\theta_2 B^2)\epsilon_t\end{aligned} \quad (10)$$

The ARIMA model in this study uses parameters (p, d, q) = (2, 1, 2), where (p = 2) is chosen based on the PACF analysis to capture patterns up to 2 lags prior, (d = 1) is based on the ADF test to achieve stationarity by removing the linear trend, and (q = 2) is derived from the ACF analysis to capture residual patterns up to 2 lags. The proposed configuration is designed to accurately capture temporal patterns and CPU usage fluctuations.

IV. Results and Discussion

This section presents the distribution fitting results, which indicate that Beta achieved the best performance based on the lowest AIC and BIC. Synthetic data based on beta testing were then used to simulate traffic over 60 minutes in Kubernetes workload testing. The performance evaluation of the model revealed that LSTM excels in prediction accuracy compared to GRU and ARIMA, thus having the potential to enhance the efficiency and stability of resource management in Kubernetes.

A. Distribution Fitting

The process of fitting distributions was performed by testing five types of distributions: Beta, Normal, Weibull, Log-Normal, and Gamma, using three main criteria. First, the Kolmogorov Smirnov (KS) test assesses the goodness of fit of the distribution to the empirical data. Second, the Akaike Information Criterion (AIC) balances the fit and the number of parameters, and the Bayesian Information Criterion (BIC) imposes an additional penalty on more complex models. This comprehensive evaluation ensures that the selected distribution accurately represents the characteristics of the data. Fig. 2 shows the demand per minute histogram with five distribution curves. It is evident that the Beta distribution with parameters a = 1.32 and b = 2.98 is the most suitable.

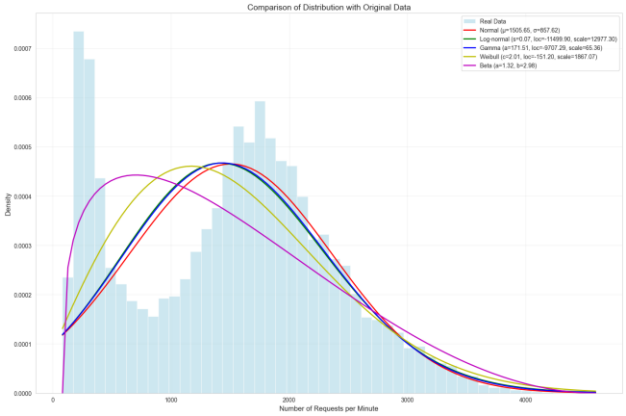

Fig. 2. Comparison of Distributions

The analysis results indicate that the Beta distribution performs best compared to other distributions, with the lowest Akaike Information Criterion (AIC) value of -4072.11 and the lowest Bayesian Information Criterion (BIC) value of -4044.84. Based on the AIC performance ranking, the Beta distribution was in first place, followed by the Weibull, gamma, log-normal, and normal distributions, as shown in Table I. These findings affirm that the Beta distribution is the most suitable for representing the data pattern, providing the best fit while maintaining model complexity.

TABLE I. Comparison of Distribution

Distribution	KS Statistic	p_value	AIC	BIC
Beta	0.121	0	-4072.1128	-4044.8412
weibull_min	0.0939	0	110021.6521	110042.1058
gamma	0.0858	0	110383.6782	110404.1319
lognormal	0.0867	0	110385.3354	110405.7891
norm	0.0835	0	110406.1863	110419.8221

B. Generate Random Variate

A Beta distribution-based variate was generated to simulate the demand data over 60 minutes, which was then used as a basis for designing load scenarios using the k6 tool. The analysis results presented in Table II indicate that the generated data exhibit characteristics that are very similar to those of the original data. This is evident from the nearly identical average values (1597.97 for synthetic data and 1505.65 for original data), comparable standard deviations (780.70 vs. 857.62), and a reasonable range of demand values per minute. In addition, the synthetic data distribution follows a Beta pattern consistent with the original data.

TABLE II. GENERATED DATA STATISTICS

Statistic	Generate Data	Original Data
Mean	1597.97	1505.65
Std Dev	780.70	857.62
Min	252	84
Max	3335	4590
Median	1491.5	1594.0
Mean	1597.97	1505.65

Data visualization in Fig. 3 through the histogram shows the similarity in distribution between the original and synthetic data, while the time series visualization illustrates a realistic demand pattern over 60 minutes, reflecting fluctuations similar to those in the actual data.

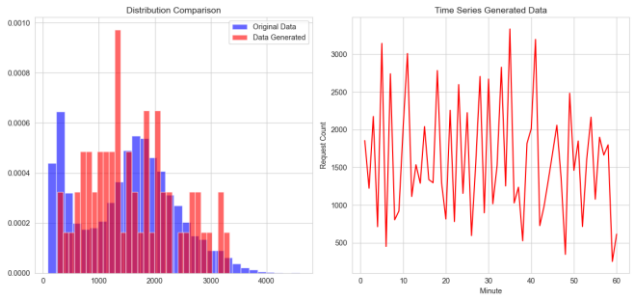

Fig. 3. Generated vs. Original Data: Distribution and Time Series

C. Generate Random Variate

Monitoring CPU usage metrics was conducted using Prometheus and Grafana, resulting in 60 data points of CPU usage over 60 minutes. Two data points, at the beginning and end, were removed because the system conditions were not stable due to the initiation and termination of traffic from k6.

Figure 4 shows the ACF values representing the correlation pattern in the data, and the PACF identifies the direct relationship between specific lags and the current data. This analysis was used to evaluate the stationarity properties of the data and determine the optimal parameters for the time series model.

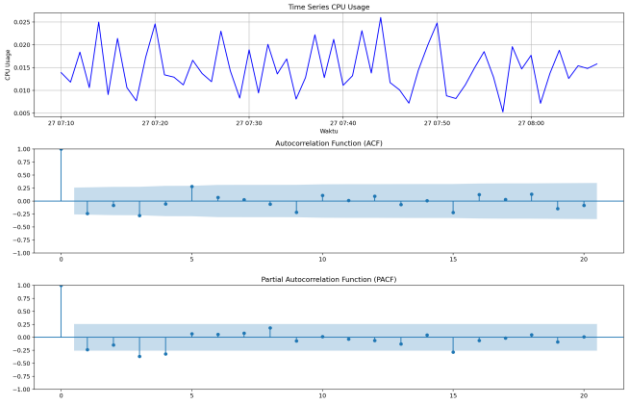

Fig. 4. CPU Usage Time Series with ACF and PACF Analysis

Figure 5 illustrates a graph that deconstructs CPU usage data into Original, Trend, Seasonal, and Residual components, which facilitates pattern analysis and modeling.

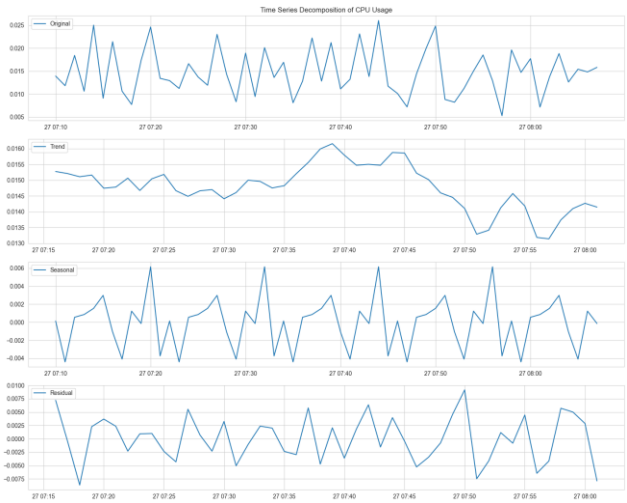

Fig. 5. Decomposition of Time Series CPU Usage

Thus, 58 stabilized data points were used for prediction. Before the predictive analysis, the stationarity of the data was evaluated using the Augmented Dickey Fuller (ADF) test. The results showed that the data is stationary, with an ADF statistic of -6.9223 and a p-value of 0.0000 (p-value < 0.05), confirming that it meets the requirements for use in a time series model.

D. Comparison of Model Performance

The performance of the ARIMA, LSTM, and GRU time series models was compared using the metrics MSE, RMSE, and R^2 to assess the accuracy of the CPU usage predictions. This analysis identifies the best model for capturing the temporal patterns of the data.

Table III shows that LSTM has the best performance with the lowest MSE, RMSE, and MAE, as well as being the only model with a positive R^2 (0.055), indicating high accuracy in CPU usage prediction. GRU performs closely

to LSTM but has a negative R^2 (-0.099), whereas ARIMA demonstrates the worst performance across all metrics. This indicates that LSTM is the most effective model for predicting CPU usage.

TABLE III. MODEL PERFORMANCE COMPARISON

Metric	ARIMA	LSTM	GRU
MSE	0.0000177133	0.0000105268	0.000012254
RMSE	0.0042087129	0.0032445064	0.0035005662
MAE	0.0029611345	0.0023938412	0.0023642956
R2 Score	-0.0559877554	0.0550595259	-0.0999774128

Figure 6 shows the training history graph, which indicates that the LSTM model has a training loss and validation loss that consistently decrease and stabilize after approximately 30 epochs, which indicates the model's ability to learn data patterns without overfitting. In contrast, the GRU model demonstrated lower training loss but showed fluctuations in validation loss, indicating a weakness in generalizability to validation data. In the Comparison of Prediction Results graph, the LSTM method produces predictions that are more accurate and closer to actual data than the GRU method, especially at peak CPU usage, where the GRU method struggles to capture extreme patterns. These results are consistent with the quantitative metrics, which demonstrate that LSTM exhibits lower MSE and RMSE, making it the best model to predict CPU usage in the context of this research.

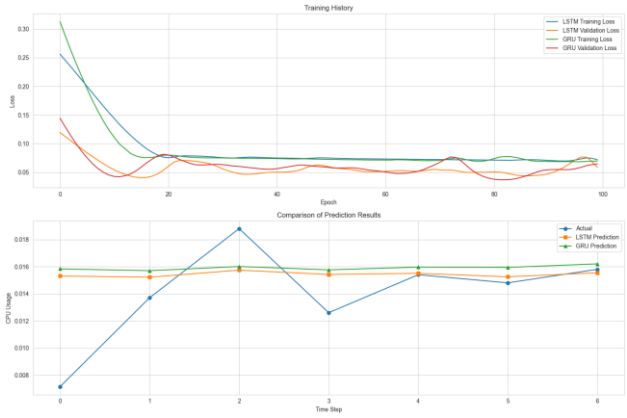

Fig. 6. Training Loss and Prediction Comparison: LSTM and GRU

Figure 7 shows the error histogram, which indicates that LSTM has a more centralized and symmetric error distribution, GRU has a slightly right-skewed distribution, and ARIMA has a wider distribution with some outliers.

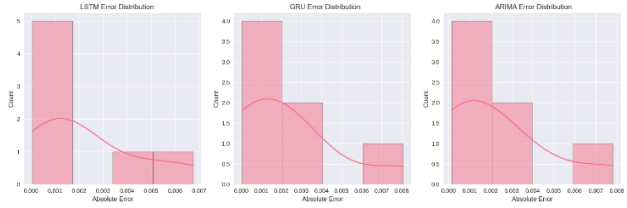

Fig. 7. Error Distribution Comparison: LSTM, GRU, and ARIMA

Table IV shows that LSTM has the most consistent error (lowest Std Error: 0.002366) and the smallest maximum error (0.006752), while GRU has the largest maximum error (0.008064), and ARIMA is in between. LSTM excels at prediction stability.

TABLE IV. STATISTICS ERROR MODEL

Model	Mean Error	Std Error	Max Error
LSTM	0.002394	0.002366	0.006752
GRU	0.002364	0.002788	0.008064
ARIMA	0.002387	0.002706	0.007807
Model	Mean Error	Std Error	Max Error

Figure 8 compares the predictions and absolute errors of LSTM, GRU, and ARIMA. The upper graph indicates that the LSTM predictions are closer to the actual data, while the lower graph shows that the LSTM errors are smaller and more consistent than the other models, emphasizing the accuracy and reliability.

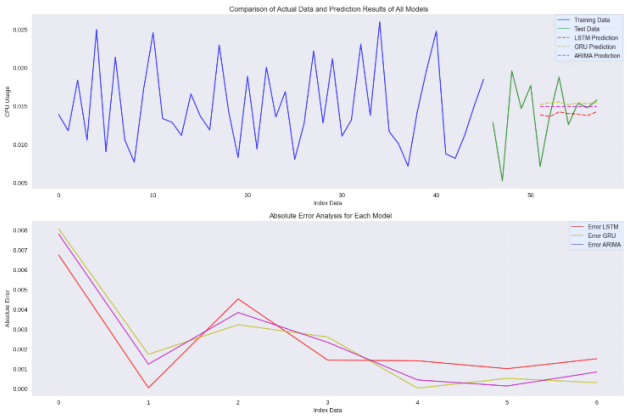

Fig. 8. Model Prediction and Error Comparison

The main finding is that LSTM has the best performance with a competitive average error (0.002394), the lowest standard deviation (0.002366), as well as the lowest MSE (0.00001053) and RMSE (0.00324451), making it the most accurate and stable model. GRU has the lowest average error (0.002364) but is less stable with a higher standard deviation (0.002788) and a negative R^2 (-0.09997741). ARIMA has a moderate average error (0.002387) but the lowest performance with the highest MSE (0.00001771) and highest RMSE (0.00420871). LSTM excels with an MSE that is 40.5% lower and an RMSE that is 22.9% lower compared to ARIMA, and it is capable of capturing complex temporal patterns, making it the best candidate for integration with Kubernetes' automatic scaling mechanisms to support real-time workload predictions.

CONCLUSION

This research shows that LSTM excels in predicting CPU usage compared to ARIMA and GRU, with the best performance across all evaluation metrics and being the only model with a positive R^2. Synthetic traffic simulations based on the Beta distribution and k6 effectively replicate realistic workload patterns. The integration of LSTM with the Horizontal Pod Autoscaler (HPA) in Kubernetes has the potential to enhance resource efficiency and application stability while also opening opportunities for the development of real-time prediction and ensemble methods. These findings contribute to resource optimization in Kubernetes.

REFERENCES

[1] S. K. Mondal et al., "Toward Optimal Load Prediction and Customizable Autoscaling Scheme for Kubernetes," Mathematics, vol. 11, no. 12, Jun. 2023, doi: 10.3390/math11122675.

[2] T. T. Nguyen, Y. J. Yeom, T. Kim, D. H. Park, and S. Kim, "Horizontal pod autoscaling in kubernetes for elastic container orchestration," Sensors (Switzerland), vol. 20, no. 16, pp. 1–18, Aug. 2020, doi: 10.3390/s20164621.

[3] F. Zaker, "Online Shopping Store - Web Server Logs," 2019, Harvard Dataverse. doi: doi/10.7910/DVN/3QBYB5.

[4] T. Nguyen et al., "An LSTM-based Approach for Predicting Resource Utilization in Cloud Computing," in The 11th International Symposium on Information and Communication Technology, New York, NY, USA: ACM, Dec. 2022, pp. 173–179. doi: 10.1145/3568562.3568647.

[5] M. El Yadari et al., "Long Short-Term Memory Networks for Server Workload Prediction," in 2024 International Conference on Intelligent Systems and Computer Vision (ISCV), IEEE, May 2024, pp. 1–7. doi: 10.1109/ISCV60512.2024.10620113.

[6] A. Saha, M. Rahman, and F. Wu, "Evaluating LSTM Time Series Prediction Performance on Benchmark CPUs and GPUs in Cloud Environments," in Proceedings of the 2024 ACM Southeast Conference on ZZZ, New York, NY, USA: ACM, Apr. 2024, pp. 321–322. doi: 10.1145/3603287.3656164.

[7] A. Bali, Y. El Houm, A. Gherbi, and M. Cheriet, "Automatic data featurization for enhanced proactive service auto-scaling: Boosting forecasting accuracy and mitigating oscillation," Journal of King Saud University - Computer and Information Sciences, vol. 36, no. 2, p. 101924, Feb. 2024, doi: 10.1016/j.jksuci.2024.101924.

[8] H. Zhou and C. H. Yong, "Implement HPA for Nginx Service Using Custom Metrics Under Kubernetes Framework," IEEE Access, vol. 12, pp. 189722–189734, 2024, doi: 10.1109/ACCESS.2024.3509876.

[9] Y. X. Chia, C. K. Seow, K. Chen, and Q. Cao, "Exploring Resource Prediction Models Based on Custom Kubernetes Auto-scaling Metrics," in 2024 9th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), IEEE, Apr. 2024, pp. 47–52. doi: 10.1109/ICCCBDA61447.2024.10569815.

[10] Y. Ding, C. Li, Z. Cai, X. Wang, and B. Yang, "A dynamic interval auto-scaling optimization method based on Informer time series prediction," IEEE Access, pp. 1–1, 2024, doi: 10.1109/ACCESS.2024.3513564.

[11] W. Miao, Y. Sun, Z. Zeng, T. Hong, and M. Xiao, "An Container Elastic Autoscaling Strategy Based Adaptive Integrated Resource Forecast," in 2024 6th International Conference on Communications, Information System and Computer Engineering (CISCE), IEEE, May 2024, pp. 525–530. doi: 10.1109/CISCE62493.2024.10653143.

[12] H. Yuan and S. Liao, "A Time Series-Based Approach to Elastic Kubernetes Scaling," Electronics (Basel), vol. 13, no. 2, p. 285, Jan. 2024, doi: 10.3390/electronics13020285.

[13] T. Li, L. Qiu, F. Chen, H. Chen, and N. Zhou, "CAROKRS: Cost-Aware Resource Optimization Kubernetes Resource Scheduler," in 2024 9th International Conference on Cloud Computing and Big Data Analytics (ICCCBDA), IEEE, Apr. 2024, pp. 127–133. doi: 10.1109/ICCCBDA61447.2024.10569976.

[14] G. Aragon, H. Puri, A. Grass, S. Chala, and C. Beecks, "Incremental Deep-Learning for Continuous Load Prediction in Energy Management Systems," in 2019 IEEE Milan PowerTech, IEEE, Jun. 2019, pp. 1–6. doi: 10.1109/PTC.2019.8810793.

[15] J. Kumar, R. Goomer, and A. K. Singh, "Long Short Term Memory Recurrent Neural Network (LSTM-RNN) Based Workload Forecasting Model For Cloud Datacenters," Procedia Comput Sci, vol. 125, pp. 676–682, 2018, doi: 10.1016/j.procs.2017.12.087.

[16] S. Nadarajah and S. Kotz, "An F 1 Beta Distribution with Bathtub Failure Rate Function," American Journal of Mathematical and Management Sciences, vol. 26, no. 1–2, pp. 113–131, Jan. 2006, doi: 10.1080/01966324.2006.10737663.

[17] M. Z. Ali, A. A. H. Alkhalidi, and A. H. Hussain, "Comparison Between Parameter Estimation for the Beta Distribution," in 2023 6th International Conference on Engineering Technology and its Applications (IICETA), IEEE, Jul. 2023, pp. 174–179. doi: 10.1109/IICETA57613.2023.10351412.

[18] J. Tacq, "The Normal Distribution and its Applications," in International Encyclopedia of Education, Elsevier, 2010, pp. 467–473. doi: 10.1016/B978-0-08-044894-7.01563-3.

[19] X. Yang, L. Xie, Y. Liu, and B. Zhao, "A Review of Parameter Estimation Methods of the Three-Parameter Weibull Distribution," in 2023 9th International Symposium on System Security, Safety, and Reliability (ISSSR), IEEE, Jun. 2023, pp. 20–31. doi: 10.1109/ISSSR58837.2023.00013.

[20] X. Yang, L. Xie, Y. Yang, B. Zhao, and Y. Li, "A comparative study for parameter estimation of the Weibull distribution in a small sample size: An application to spring fatigue failure data," Qual Eng, vol. 35, no. 4, pp. 553–565, Oct. 2023, doi: 10.1080/08982112.2022.2158745.

[21] W. Thangjai, S.-A. Niwitpong, S. Niwitpong, and N. Smithpreecha, "Confidence Interval Estimation for the Ratio of the Percentiles of Two Delta-Lognormal Distributions with Application to Rainfall Data," Symmetry (Basel), vol. 15, no. 4, p. 794, Mar. 2023, doi: 10.3390/sym15040794.

[22] R. Heijungs, "Analysis and remediation of the confusing specification of the lognormal distribution," Int J Life Cycle Assess, vol. 29, no. 3, pp. 537–554, Mar. 2024, doi: 10.1007/s11367-023-02249-8.

[23] J. Harvey and A. J. van der Merwe, "Bayesian confidence intervals for means and variances of lognormal and bivariate lognormal distributions," J Stat Plan Inference, vol. 142, no. 6, pp. 1294–1309, Jun. 2012, doi: 10.1016/j.jspi.2011.12.006.

[24] A. A. Bromideh and R. Valizadeh, "Discrimination between Gamma and Log-Normal Distributions by Ratio of Minimized Kullback-Leibler Divergence," Pakistan Journal of Statistics and Operation Research, vol. 9, no. 4, p. 443, Feb. 2014, doi: 10.18187/pjsor.v9i4.487.

[25] S. Nadarajah, "The gamma beta ratio distribution," Braz J Probab Stat, vol. 26, no. 2, May 2012, doi: 10.1214/10-BJPS128.

[26] A. Murari, E. Peluso, F. Cianfrani, P. Gaudio, and M. Lungaroni, "On the Use of Entropy to Improve Model Selection Criteria," Entropy, vol. 21, no. 4, p. 394, Apr. 2019, doi: 10.3390/e21040394.

[27] J. Kuha, "AIC and BIC," Sociol Methods Res, vol. 33, no. 2, pp. 188–229, Nov. 2004, doi: 10.1177/0049124103262065.

[28] P. C. Emiliano, M. J. F. Vivanco, and F. S. de Menezes, "Information criteria: How do they behave in different models?," Comput Stat Data Anal, vol. 69, pp. 141–153, Jan. 2014, doi: 10.1016/j.csda.2013.07.032.

[29] M. Forster and E. Sober, "Aic Scores as Evidence," in Philosophy of Statistics, Elsevier, 2011, pp. 535–549. doi: 10.1016/B978-0-444-51862-0.50016-2.

[30] G. Song, L. Zhu, A. Gao, and L. Kong, "Blockwise AICc and its consistency properties in model selection," Commun Stat Theory Methods, vol. 50, no. 13, pp. 3198–3213, Jul. 2021, doi: 10.1080/03610926.2019.1691734.

[31] R. A. Andhika Viadinugroho and D. Rosadi, "Long Short-Term Memory Neural Network Model for Time Series Forecasting: Case Study of Forecasting IHSG during Covid-19 Outbreak," J Phys Conf Ser, vol. 1863, no. 1, p. 012016, Mar. 2021, doi: 10.1088/1742-6596/1863/1/012016.

[32] Z. Zhang, Q. Chen, T. Han, C. Li, Y. Liu, and G. Liu, "Memristor-Based Circuit Demonstration of Gated Recurrent Unit for Predictable Neural Network," IEEE Trans Electron Devices, vol. 69, no. 12, pp. 6763–6768, Dec. 2022, doi: 10.1109/TED.2022.3217116.

[33] S. Mohsen, "Recognition of human activity using GRU deep learning algorithm," Multimed Tools Appl, vol. 82, no. 30, pp. 47733–47749, Dec. 2023, doi: 10.1007/s11042-023-15571-y.

[34] M. C. Pandey and P. S. Rawat, "Virtual Machine Provisioning Within Data Center Host Machines Using Ensemble Model in Cloud Computing Environment," SN Comput Sci, vol. 5, no. 6, p. 690, Jul. 2024, doi: 10.1007/s42979-024-03056-0.

S. S, N. S, and S. V. D. K, "Time Series Forecasting of Cloud Resource Usage," in 2021 IEEE 6th International Conference on Computing, Communication and Automation (ICCCA), IEEE, Dec. 2021, pp. 372–382. doi: 10.1109/ICCCA52192.2021.9666444.